

Basic (your first real API call) — Hints

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Only 2 hints — work through them in order, and don't jump ahead until you've genuinely tried. Each hint has 2 depth levels: **Basic** (the plain idea) and **Intermediate** (proper Python). Read Basic first even if you already know Python — it's the fastest way to spot exactly what Intermediate adds.

- [Hint 1 — The idea, and the exact pieces](#)
- [Hint 2 — The plan, and almost the whole thing](#)

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Hint 1 — The idea, and the exact pieces

Basic Version

You're doing two separate things here: make one API call that works, then make one API call that's broken on purpose.

The `requests` library gives you a `.get(url)` function. Call it on a real, working address, and look at what comes back — it has a `.status_code` (a number like 200) and a `.json()` method that turns the reply into a Python dictionary.

Then break the URL on purpose — a typo in the protocol, like `htp://` instead of `http://` — and see what Python does differently. It won't give you back a response at all; it'll raise an error before it even reaches the network.

Things to use:

- `requests.get("https://api.github.com")` — makes the call.
- `.status_code` — the number, like `200`.
- `.json()` — turns the body into a Python dict.
- `try: ... except Exception as e: print(type(e).__name__)` — catches whatever goes wrong and prints its name.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

The exercise is really about seeing two different failure categories with your own eyes, not just reading about them.

A **successful call** gives you a `requests.Response` object — `.status_code: int` and `.json() -> dict | list` (it parses the body as JSON for you, raising `json.JSONDecodeError` if the body isn't valid JSON).

A **malformed URL** never reaches the network at all — `requests` raises `requests.exceptions.MissingSchema` (or a similar `requests.exceptions.RequestException` subclass) immediately, client-side, before any bytes go anywhere. This is a completely different kind of failure than a bad *response* — nothing was sent, so there's nothing to receive.

The exact pieces:

- `requests.get(url) -> requests.Response` — the return type is always a `Response` object on success, never `None`.
- `requests.exceptions.RequestException` — the base class for everything `requests` can raise client-side (bad URL, connection refused, timeout, etc.). Catching this specific base — not bare `Exception` — is the right habit once you get past this exercise.
- `type(e).__name__` — every exception instance knows its own class; this is how you inspect exactly what you caught without guessing.

Write both blocks — the working call and the broken one — before checking Hint 2.

Difference between Basic and Intermediate: Basic names the tools for the two obvious outcomes (a working call, a broken URL). Intermediate names the real exception class involved and the habit of catching it specifically — this is the depth the exercise's Solution is written at. Telling apart the several distinct kinds of client-side failure (DNS, connection refused, a 4xx/5xx status) is exactly what this document's Build Task does properly, in its own exercises below.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Hint 2 — The plan, and almost the whole thing

Basic Version

```
call requests.get on a real URL
print the status code
print the parsed JSON

try:
    call requests.get on a broken URL
except any error as e:
    print the error's class name
```

Here's almost the whole thing:

```
# api_first_call_practice.py
import requests

response = requests.get("https://api.github.com")
print(response.status_code)
```

Expected output if you run just this: a number like `200` , printed once. What's missing: printing `response.json()` , and the whole second block with the broken URL and `try/except` . Add those yourself, then check Hint's Intermediate Version below.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

```
import requests

response = requests.get("https://api.github.com")
print(response.status_code)
print(response.json())

try:
    requests.get("http://broken")
except requests.exceptions.RequestException as e:
    print(type(e).__name__, "-", e)
```

```
# api_first_call_practice.py
import requests

response = requests.get("https://api.github.com")
print(response.status_code)
print(response.json())

try:
    requests.get("http://broken")
except requests.exceptions.RequestException as e:
    ... # what goes here?
```

Fill in the `except` block yourself, then compare against the [Solution](#). Notice the working call needs no `try/except` at all for this exercise — you're only wrapping the call you already know will fail, to see exactly how it fails.

Difference between Basic and Intermediate: same idea (try a call, watch it fail) at 2 completeness levels — Basic proves a call works and a broken one raises something. Intermediate names the specific exception class and catches only that.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Full solution: [Show me the solution](#)